

Taller KNN

Estudiantes

Lenin Enrique García Caraballo

David Orlando Riaño Morales

Docente

Alexander Pérez Ruiz

Escuela Colombiana de Ingeniería Julio Garavito

Métodos computacionales en ingeniería – MCEI

2026-2

26 de octubre de 2026

1 Preguntas exploratorias

1.1 ¿Qué diferencias conceptuales encuentras entre un método basado en distancias como KNN y una estructura espacial como el kd-tree?

El KNN es un algoritmo de aprendizaje de clasificación, el cual evalúa un dato calculando la distancia desde su posición hasta todos los demás puntos del conjunto de datos, para así saber a que subconjunto pertenece.

En cambio, el kd-tree es una estructura de datos espaciales que particiona el espacio de manera jerárquica mediante un árbol binario. En lugar de evaluar cada punto, el algoritmo descarta ramas enteras del espacio geométrico que no pueden contener el vecino más cercano, reduciendo su complejidad de búsqueda.

1.2 ¿En qué escenarios prácticos crees que KNN puede ser más eficiente que un método basado en kd-tree, y viceversa?

El algoritmo KNN suele ser más eficiente cuando se trabaja con muchas variables, pocos datos o conjuntos de datos que cambian constantemente. Esto se debe a que calcula las distancias directamente entre los datos, sin necesidad de construir una estructura adicional. Aunque este proceso puede ser lento cuando hay muchos registros, es útil cuando hay pocas muestras o cuando los datos se actualizan frecuentemente. Además, cuando hay muchas variables, un kd-tree puede perder eficiencia y terminar revisando casi todos los datos de todas formas.

El kd-tree es más eficiente cuando hay pocas variables y muchos datos. Esta estructura organiza los datos en forma de árbol, lo que permite encontrar los vecinos más cercanos sin tener que comparar cada registro individualmente. Por ejemplo, en el ejercicio de aplicación, donde se trabaja con voltaje, corriente y potencia, un kd-tree puede ser útil si se tienen muchos registros y se necesitan realizar muchas consultas. Aunque construir el árbol requiere un tiempo inicial, este esfuerzo se compensa cuando se realizan muchas búsquedas.

1.3 ¿Qué impacto tienen la dimensionalidad y el tamaño de los datos en el desempeño de estos métodos?

Cuando se trabaja con muchas variables, encontrar los vecinos más cercanos se vuelve más difícil. En el caso del kd-tree, al aumentar el número de variables, pierde eficiencia porque necesita revisar más partes del árbol y puede terminar evaluando casi todos los registros. KNN tradicional también se ve afectado, ya que las distancias entre los datos tienden a ser más parecidas, lo que dificulta identificar cuáles son realmente los más cercanos.

En cuanto a la cantidad de datos, KNN tradicional se vuelve más lento a medida que aumenta el número de registros, porque debe calcular la distancia entre el dato consultado y cada uno de los registros existentes. En cambio, el kd-tree permite realizar búsquedas más rápidas cuando se trabaja con pocas variables y muchos registros, ya que organiza los datos en un árbol que facilita encontrar los vecinos sin revisar todos los elementos. Aunque construir esta estructura toma tiempo al principio, es útil cuando se necesitan realizar muchas consultas.

1.4 ¿Cómo influye la elección de la métrica de distancia (euclidiana, Manhattan, etc.) en los resultados de KNN?

Influye bastante, porque dependiendo del la métrica de distancia elegida, cambia el valor de la distancia. Por ejemplo, con la métrica euclidiana la distancia que se busca calcular es la longitud de la línea recta (hipotenusa) entre dos puntos, mientras que si se usa la distancia Manhattan, se

está calculando la distancia vertical y horizontal entre esos dos puntos. Claramente, son distancias diferentes y esto hace que se alteren directamente las fronteras de decisión y las predicciones de clasificación.

1.5 ¿Qué limitaciones prácticas podrían surgir al implementar estos métodos en sistemas de ingeniería eléctrica de gran escala?

El tiempo de procesamiento aumenta cuando hay muchos datos. En una red eléctrica con muchos sensores que envían información constantemente, KNN tradicional puede volverse lento porque debe comparar cada nuevo dato con todos los registros existentes. El kd-tree también puede perder eficiencia cuando se analizan muchas variables eléctricas al mismo tiempo, como voltajes, corrientes, potencias y temperaturas.

También, los datos cambian constantemente y es necesario actualizar la información. En una red eléctrica, el consumo de energía varía y pueden ocurrir cambios y fallas en el sistema. Mantener actualizado un kd-tree puede requerir mucho procesamiento y es un problema en sistemas de protección y control que necesitan responder en poco tiempo.

Otro problema es el ruido y los errores en las mediciones. Los sensores eléctricos pueden registrar valores incorrectos debido a interferencias, ruido eléctrico o falta de calibración. Como KNN y kd-tree utilizan distancias para comparar los datos, estas mediciones pueden afectar los resultados y provocar falsas alarmas o diagnósticos equivocados.

La capacidad limitada de los dispositivos electrónicos dificulta su implementación. Algunos equipos utilizados para procesar datos cerca de los sensores tienen poca memoria y capacidad de procesamiento. Almacenar grandes cantidades de información o ejecutar estos algoritmos directamente en ellos puede ser complicado y afectar su funcionamiento.

2 Ejercicio de aplicación

2.1 Utilice los datos del archivo cvs adjunto que contiene 100 sensores, cada uno con tres variables: voltaje, corriente y temperatura, para aplica el método KNN e identificar los 5 vecinos más cercanos de un sensor que presenta una anomalía en la medición de voltaje.

Primero, se importaron las bibliotecas necesarias para realizar el análisis: NumPy y Pandas para el manejo de datos, NearestNeighbors de Scikit-learn para aplicar el algoritmo KNN y StandardScaler para normalizar las variables. Posteriormente, se cargó el archivo Datos_MCEI.csv mediante Pandas, utilizando el punto y coma como separador de columnas.

Después, se seleccionaron las variables de voltaje, corriente y potencia aproximada para realizar el análisis de proximidad entre los sensores. Estas características se almacenaron en una matriz llamada X la cual es la entrada del algoritmo KNN. Después, se normalizaron los datos con StandardScaler para que las variables de mayor escala (como potencia o voltaje) no dominen sobre la corriente en el cálculo de distancias.

Para identificar el sensor anómalo, se buscó el valor mínimo de la columna voltaje_V con la función idxmin(). Este procedimiento identifica el sensor con el menor voltaje registrado, asumiendo que dicha condición corresponde a la anomalía planteada.

Después de identificar el sensor anómalo, se implementó el algoritmo KNN utilizando la clase NearestNeighbors y la distancia euclidiana para medir la proximidad. Se eligió un valor de $k = 5$ (los cinco vecinos más cercanos). Sin embargo, en el código se eligieron 6 vecinos, ya que el primer resultado corresponde al propio sensor anómalo, donde su distancia respecto a sí mismo es cero.

Por último, se excluyó el propio sensor anómalo de los resultados y se recorrieron los cinco vecinos restantes para imprimir su ID, voltaje, corriente, potencia aproximada y distancia euclidiana normalizada. Cuanto menor sea la distancia, más cercanas serán sus características.

A continuación se muestran los resultados:

```
Sensor anómalo detectado:
Muestra ID: 74
Voltaje: 1181 V
Corriente: 507 A
Potencia Aprox: 60129 W

Los 5 vecinos más cercanos al sensor anómalo:

Vecino 1:
  - Muestra ID: 94
  - Voltaje: 1250 V
  - Corriente: 491 A
  - Potencia Aprox: 61514 W
  - Distancia Euclidiana (Normalizada): 0.1542

Vecino 2:
  - Muestra ID: 48
  - Voltaje: 1188 V
  - Corriente: 488 A
  - Potencia Aprox: 57271 W
  - Distancia Euclidiana (Normalizada): 0.2615

Vecino 3:
  - Muestra ID: 27
  - Voltaje: 1193 V
  - Corriente: 572 A
  - Potencia Aprox: 68055 W
  - Distancia Euclidiana (Normalizada): 0.7659

Vecino 4:
  - Muestra ID: 86
  - Voltaje: 1203 V
  - Corriente: 409 A
  - Potencia Aprox: 49947 W
  - Distancia Euclidiana (Normalizada): 1.0382

Vecino 5:
  - Muestra ID: 47
  - Voltaje: 1233 V
  - Corriente: 587 A
  - Potencia Aprox: 71778 W
  - Distancia Euclidiana (Normalizada): 1.0740

Tiempo de ejecución (KNN): 0.4532 ms
```

Figura 1: Resultados de 5 vecinos más cercanos a sensor anómalo con KNN

2.2 Construye una estructura kd-tree para organizar espacialmente los sensores y utiliza esta estructura para encontrar los vecinos más cercanos del mismo sensor.

Primero, se importaron las bibliotecas necesarias para realizar el análisis: NumPy y Pandas para el manejo de datos, NearestNeighbors de Scikit-learn para implementar la búsqueda de vecinos más cercanos y StandardScaler para normalizar las variables. Posteriormente, se cargó el archivo

Datos_MCEI.csv mediante Pandas, utilizando el punto y coma como separador de columnas.

Se siguió el mismo procedimiento descrito para el método KNN y después de identificar el sensor anómalo, se construyó una estructura espacial kd-tree utilizando la clase NearestNeighbors, con el parámetro algorithm='kd_tree'. Esta estructura organiza los datos de acuerdo con sus características y permite realizar búsquedas de vecinos cercanos sin tener que calcular la distancia entre el sensor analizado y todos los demás sensores en cada consulta. Para determinar la proximidad, se utilizó la distancia euclidiana sobre los datos normalizados.

Para realizar la búsqueda, primero se ajustó la estructura kd-tree con los datos normalizados y luego se consultaron los vecinos más cercanos al sensor anómalo.

A continuación se muestran los resultados:

```
Sensor anómalo detectado:
Muestra ID: 74
Voltaje: 1181 V
Corriente: 507 A
Potencia Aprox: 60129 W

Los 5 Vecinos Más Cercanos al Sensor Anómalo (kd-tree):

Vecino 1:
  - Muestra ID: 94
  - Voltaje: 1250 V
  - Corriente: 491 A
  - Potencia Aprox: 61514 W
  - Distancia Euclidiana (Normalizada): 0.1542

Vecino 2:
  - Muestra ID: 48
  - Voltaje: 1188 V
  - Corriente: 488 A
  - Potencia Aprox: 57271 W
  - Distancia Euclidiana (Normalizada): 0.2615

Vecino 3:
  - Muestra ID: 27
  - Voltaje: 1193 V
  - Corriente: 572 A
  - Potencia Aprox: 68055 W
  - Distancia Euclidiana (Normalizada): 0.7659

Vecino 4:
  - Muestra ID: 86
  - Voltaje: 1203 V
  - Corriente: 409 A
  - Potencia Aprox: 49947 W
  - Distancia Euclidiana (Normalizada): 1.0382

Vecino 5:
  - Muestra ID: 47
  - Voltaje: 1233 V
  - Corriente: 587 A
  - Potencia Aprox: 71778 W
  - Distancia Euclidiana (Normalizada): 1.0740

Tiempo de ejecución (kd-tree): 0.4721 ms
```

Figura 2: Resultados de 5 vecinos más cercanos a sensor anómalo con kd-tree

2.3 Compara los resultados obtenidos con ambos métodos: tiempo de ejecución, precisión en la identificación de vecinos y facilidad de interpretación.

2.3.1. Tiempo de ejecución

Teniendo en cuenta los tiempos registrados en las imágenes 1 y 2, puede verse que el método **KNN** fue más rápido que el de **kd-tree** por 0.0189 milisegundos. Debido a que esta diferencia de tiempos es muy pequeña, se decidió ejecutar varias veces los códigos y registrar los tiempos entre cada uno para poder identificar si siempre se da el caso en que **KNN** es un poco más

rápido de **kd-tree**:

No.	Tiempo KNN [ms]	Tiempo kd-tree [ms]	Diferencia [ms]	Ganador
1	0,3242125	0,2952526	0,02896	kd-tree
2	0,3196303	0,2636335	0,055997	kd-tree
3	0,3228442	0,3059184	0,016926	kd-tree
4	0,2979275	0,2900687	0,007859	kd-tree
5	0,2936733	0,3005936	0,00692	KNN
6	0,325566	0,2633255	0,062241	kd-tree
7	0,3218697	0,2795595	0,04231	kd-tree
8	0,3021111	0,2960651	0,006046	kd-tree
9	0,3310187	0,2835769	0,047442	kd-tree
10	0,4605288	0,3491248	0,111404	kd-tree

Teniendo en cuenta la tabla anterior, el **kd-tree** fue un poco más rápido que el **KNN** en casi todos los casos. Esto puede deberse a que el KNN por fuerza bruta debe calcular cada distancia entre el sensor anómalo y los otros 99 sensores. En cambio **kd-tree**, utiliza una estructura que puede descartar ramas enteras del árbol. **kd-tree** ignora los conjuntos de sensores que sabe que estan espacialmente lejanos, lo que reduce el trabajo del procesador.

Otra razón de por que **kd-tree** es más rápido, es porque se está trabajando con pocas dimensiones en el espacio de características (Voltaje, Corriente y Potencia Aproximada). Los **kd-trees** tienen un mejor rendimiento cuando hay pocas variables.

2.3.2. Precisión en la identificación de vecinos

Al revisar las imágenes 1 y 2, puede observarse que la precisión es la misma, debido a que coinciden exactamente con los mismos resultados. Debido a que ambos algoritmos están implementados en **scikit-learn**, realizan una búsqueda exacta y no una estimación.

De esto, puede concluirse que el uso de una estructura avanzada basada en decisiones como **kd-tree** para particionar geométricamente el espacio de los sensores no implica una pérdida de precisión frente a un calculo exhaustivo de todas las distancias entre el sensor anómalo y los demás sensores que hace el **KNN**. Las distancias hasta el cuarto decimal (0.1542 a 1.0740) demuestran que el algoritmo descartó de manera segura a los 94 sensores restantes basándose en umbrales matemáticos garantizados, no en estimaciones.

2.3.3. Facilidad de interpolación

- **KNN (Fuerza Bruta):** Es el método más fácil de interpretar debido a que consiste simplemente en calcular la distancia entre el sensor anómalo elegido y los otros 99 sensores, para luego ordenar las distancias de menor a mayor, y elegir los 5 vecinos más cercanos (que serían otros sensores anómalos).
- **kd-tree:** Es más complejo de interpretar porque introduce jerarquías. Para entender cómo el algoritmo llega al resultado, debe imaginarse el espacio de características dividido recursivamente por hiperplanos construidos sobre las medianas de voltaje, de corriente y sobre las medianas de potencia. El algoritmo descarta sectores de datos delimitados por los hiperplanos empleando árboles de decisión binarios. Lo anterior hace que **kd-tree** sea una herramienta menos intuitiva.

2.4 Discute cúal método sería más adecuado para un sistema de monitoreo en tiempo real en ingeniería eléctrica

Elegir el método más adecuado depende de que tan vólatiles sean los datos del sistema:

- **kd-tree:** Si los sensores evalúan su estado en tiempo real comparándose contra una base de datos estática gigante (por ejemplo, miles de patrones históricos de fallas y operación nominal), el kd-tree es mejor que el KNN. El modelo del árbol de decisión se construye una sola vez en el preprocesamiento, permitiendo que las consultas en tiempo real se hagan en tiempos muy cortos gracias a su complejidad algorítmica logarítmica.
- **KNN:** Si el sistema de referencia no es estático y los valores de todos los nodos cambian constantemente en cada instante de tiempo, la estructura del kd-tree no sería muy útil porque habría que construir el modelo del árbol por cada cambio en los datos. Reconstruir el árbol continuamente introduciría un cuello de botella inaceptable. En este escenario, el cálculo matricial directo de KNN resulta más eficiente al no requerir tiempos de ensamblaje previo, sino que por cada cambio simplemente se calcularían las distancias euclidianas y se organizarían los datos para encontrar a los vecinos más cercanos, lo que es mucho más simple que construir en cada instante un modelo de árbol.